CS5013 Principles & Practices

Exchange Student Guide & Community Platform

Mikhail Novikov, GE26Z858 Abdirakhim Ismailov, GE26Z860

11 October 2026

Repository: https://github.com/rikire/Exchange-Student-Guide-Platform. The code is written with Claude Code. What follows is how we keep that honest: each principle names the file or mechanism that enforces it and a dated case from the history where it mattered. The prompt journal (`docs/ai/journal/`, 1126 entries since 3 September) holds every prompt quoted here.

1 Prompt engineering

Context in layers, not one long prompt. Standing rules live in files the assistant loads itself: `CLAUDE.md` (six rules and where to look), `docs/ai/*.md` (one topic each: prompting, testing, when to stop) and `.claude/rules/` (the short part that must be in view while code is written). A prompt then carries only the task. A rule read once at the start of a session loses to everything said later, so a `UserPromptSubmit` hook re-injects the core rule with every prompt.

A contract before any change. For a request that changes files or state, the assistant first reads what the repository can answer, then states the goal, the boundaries, acceptance criteria phrased so they can become test names, open questions *each with a suggested answer*, and steps each with its check — and waits for an explicit "yes" (`docs/ai/prompting.md`). "Propose and continue" is not agreement; choosing a default and announcing it is not compliance. The confirmed sentences become the acceptance criteria and the test names, so one sentence runs from the prompt to the assertion. Appendix A.1 and A.3.

Closed questions with a recommendation. A question to the human offers options, the consequence of each, and which one we would pick, so the answer can be one word ("A", "yes"). Facts the repository holds are looked up, never asked.

Make the model verify, not recall. Versions are checked against Maven Central or npm before they are written down; a claim about a library is checked in its source. Saying "not verified" is allowed; a plausible guess is not. When the human challenged a claim ("what is this ban on JavaScript?", A.2), the answer was a search of the requirements, not a defence of the claim.

Reusable prompts as skills. Recurring tasks are slash commands in `.claude/skills/`: `/sharpen` writes the contract without acting, `/feature`, `/gaps`, `/dod`, `/trace-check`.

2 Debugging

We follow the order in `docs/ai/stop-and-ask.md`: reproduce, form one hypothesis that a check can refute, test it, and fix with a test that was red first. Observable triggers stop the work: a check that refutes a hypothesis, a second attempt with nothing new learned, a result that contradicts a requirement. After a stop we write down what is confirmed, refuted and unknown, and trace everything that rested on the wrong premise.

A case: the moderator's approval answered 500 (5 October). Seen only in a rehearsal of the demo on the Docker stand. *Evidence*: the application log, not the page. *Hypothesis*: the search index on the stand's volume predated a change to its fields. *Check*: the database held the approval while search did not find it, and the index builder skipped any index that held documents. *Fix*: rebuild the index at every start, an ADR amended with the human's decision, and a test inverted to state it. The same rehearsal found three more faults no unit test could see, each fixed with the test that would have caught it.

Read the library, then prove the reading. Before adding an image-preview plugin we read its bundle and found it loads images from `blob:` URLs, which our Content-Security-Policy blocks (A.3).

After widening the policy we restored the old one on purpose and watched the browser test go red with the browser's own CSP message, so the test is known to catch it.

Do not close what is not proven — then prove it. A tag-field browser test failed in 2 of 5 full runs and never alone. A first fix removed a 300 ms refresh delay found in Tom Select's source, but no test went red without it, so `DEBT-023` stayed *open*, with that written down. Logging during the test changed the timing and hid the fault, so we recorded the field's state only *after* a failure: the tag list had reopened with a stored tag lit. Reading the library again found the mechanism — `open()` calls `focus()`, whose handler reopens the list a moment later — and the same fault was then reproduced by hand in Chrome on the stand. One setting (`openOnFocus: false`) fixed it: 0 failures in 10 runs. The cause was in the product, not the test: a fast typist could have got the wrong tag.

3 Version control: code and prompts

The prompts are in Git. Hooks write every prompt, an English rendering when it was not in English, the outcome, the human's own edits between turns and the checks that passed, into `docs/ai/journal/`; the `Stop` hook commits the entry. The author is resolved from the Git identity. The original prompt is kept beside the translation: it is the artefact.

The journal is tested like the code. Choosing the sample prompts below, we found two entries whose English rendering belonged to a different prompt: a prompt sent while a turn was running overwrote the rendering of the first. That is `DEBT-027`: four `JournalTest` cases red first, then one rendering per prompt in order. The spoiled entries were not rewritten — a note in the journal of 9 October corrects them, because the journal is a record, not a draft.

Commits say what they deliver. `<type>(<scope>): <subject> [FR-…]`; a `feat` or `fix` must name its feature and requirement, checked by the `commit-msg` hook. Code carries `//trace:FR-XXX` anchors, so `docs/traceability.md` is generated from code, tests and commits, and a requirement with no test shows as a gap.

Checks that cannot be skipped. `pre-commit`: Spotless and fast tests; `pre-push`: the full build plus "generated documents current" and "a changed controller is described in the route contract"; CI repeats it and runs the suite on PostgreSQL. The client refuses `--no-verify` and `-DskipTests` outright. On 2 October a push was refused three times — stale traceability, stale gap list, an undocumented controller change — and each was fixed, not bypassed. Unpushed work is rebased onto the partner's (`pull.ff=only`), and conflicts in generated files are resolved by running the generator, never by hand.

4 Testing, refactoring and documentation

Test first. Interfaces, a red test, the minimal code, then refactor (`docs/ai/workflow.md`); a test green before the change is reported as a guard, not as evidence. Test names are sentences (`a_refused_file_is_marked_beside_the_file_field_and_must_be_chosen_again`). Today: 638 application tests and 356 for the tools; 151 browser tests in Chromium with axe at four widths; query-count tests that fail when a page's queries grow with its rows; mutation testing (PIT, 130 of 132 mutants killed, the two survivors explained); property-based tests (jqwik, 1000 generated titles in English, Hindi and Tamil).

Refactoring toward one place for each rule. When the tag field needed to compare typed tags with stored ones, the normalisation rule was moved out to `Tags.storedAs` rather than copied: a copy of that rule had already been debt once (`DEBT-005`). Spring Modulith and ArchUnit tests keep the thirteen slices from reaching into each other's internals.

Documentation that cannot drift. Changed behaviour updates its document in the same commit (`docs/ai/docs-sync.md`); a `PostToolUse` hook names the document an edit puts out of date. Four documents are generated and refused at push when stale: traceability, the gap list, the route tables, slice ownership. Decisions are ADRs (22, each with the options weighed); anything temporary is a `DEBT` entry with cause, consequence, fix and trigger, and a `TODO` without one is refused.

Appendix A. Sample prompts, from the journal

Each prompt is quoted as typed (Russian), with the English rendering the journal keeps and an abridged outcome.

A.1 — A short prompt, a contract before state changes (2026-10-02, 14:50)
Prompt: "сделай пулл, проверь что сделано и что нужно сделать" — *Do a pull, check what has been done and what needs to be done.*
Outcome: fetched 125 commits; did not pull, since the branch had diverged and `pull.ff=only` would refuse; proposed `pull --rebase --autostash` and waited. *Next prompt:* "да, делай rebase" — *Yes, do the rebase.*

A.2 — Challenging the model's claim (2026-10-02)
Prompt: "погоди, а что за запрет на js? откуда он вообще и зачем" — *Wait, what is this ban on JS? Where does it come from and why?*
Outcome: checked the requirements and ADRs: nothing forbids JavaScript; the "rule" was one descriptive sentence in a feature file. The assistant said it had overstated it and changed its recommendation.

A.3 — A stop on a trigger (2026-10-02)
Prompt: "1. давай твой вариант. начинай" — *Go with your option. Start.*
Outcome: reading the image-preview plugin's bundle against the CSP refuted the assumption that it needed no policy change. The assistant stopped before writing the ADR and asked: widen the policy by `img-src blob:` and `worker-src blob:`, or drop the preview. The human chose the first.

A.4 — A contract for a multi-step change (2026-10-02)
The plan for the submission forms, as the assistant proposed it and the human confirmed: seven steps (an ADR; the tag field; the drop zone; the editor; errors at the field; the draft; the edit form), each with its check, one commit each, "each step in turn, together". Open questions came with a suggested answer, e.g. *exclude the library's transitive WebJars? — suggested: yes, the bundle contains them.*

A.5 — A debugging prompt (2026-10-05)
Prompt: a screenshot of *HTTP Status 500* and "после аппрува" — *after the approval.*
Outcome: read the stand's log; found the approval saved and the index write refused by a mapping from an older version; offered three ways forward with a recommendation; the human answered "1А, 2 да, 3 да, 4 да" — *1A, 2 yes, 3 yes, 4 yes.*